

Relaytic-AML: A Local-First Evaluation Lab for Financial-Crime Machine Learning

Author Name
Affiliation
`contact@example.com`

June 2026

Abstract

Financial-crime machine learning is usually judged by detector scores, but operational anti-money laundering (AML) work depends on evidence a team can trust: data posture, temporal validity, graph provenance, review capacity, and claim discipline. Relaytic-AML is a local-first evaluation lab for that setting. It is built as a set of cooperating specialist agents that share a governed local artifact graph: a guide for orientation, a scout for source and leakage checks, a strategist for task contracts, a scientist for challenge, a builder for bounded execution, trace reviewers for auditability, and release governors for public claims.

Relaytic began as a general local inference-engineering system. In this work it is sharpened into an AML-focused environment where a human or external agent can ask what is known, which artifacts matter, and what should happen next. The public evidence includes supporting PaySim synthetic temporal-fraud precision-recall area under the curve (PR-AUC) 0.638773, supporting Elliptic temporal graph-feature PR-AUC 0.668756, and Elliptic2 modern-context PR-AUC 0.94324 $\pm$ 0.000882, below the recorded RevClassifyDS reference of 0.974. The main contribution is a reproducible architecture for keeping experiments, review assumptions, limitations, and reported claims aligned while private data remains local by default.

1 Introduction

Anti-money-laundering systems are operational decision systems. A useful model does not only rank transactions or entities. It has to respect time, explain why a case reached a review queue, preserve graph or entity provenance, and make clear what a result does not prove. This is especially hard in financial-crime work because the most realistic data is private, licensed, regulated, or operationally sensitive.

That privacy boundary creates a scientific problem as much as an engineering problem. Public AML research often has to work with synthetic data, anonymized transaction graphs, or partial blockchain views, while real production teams care about delayed labels, changing behavior, investigation budgets, typologies, and model-risk review. A benchmark row is therefore incomplete unless the reader can see the data contract, the split rule, the operating point, and the reason a stronger interpretation is or is not allowed.

Relaytic was first built as a broader local inference lab for structured data. The AML edition is a deliberate narrowing of that idea. Instead of trying to be a generic benchmark runner, Relaytic-AML asks a more practical question: can a local machine or local server become a trustworthy evaluation lab where data, modeling work, agent assistance, review assumptions, and public claims stay connected?

The answer explored here is architectural. Relaytic-AML treats the local workspace as the authority. Models, tables, context packs, figures, and paper text are downstream of artifacts on disk. Agents are useful, but they act through bounded roles. A guide explains the current state. A scout checks data posture and leakage risk. A scientist challenges the modeling plan. A builder runs the experiment. A claim governor decides whether a result can be described publicly. The user is never expected to know which file to open first because Relaytic itself can explain the run state and export a compact context pack for another large language model (LLM) or coding agent.

The benchmarks in this paper are deliberately modest in their role. They are stress tests for the environment. A score is valuable only if the reader can see the dataset boundary, split rule, budget, operating point, limitation, and claim boundary that produced it. This paper therefore presents Relaytic-AML as an evaluation lab first and a benchmark package second.

2 Contribution and Scope

This paper is a systems paper about local-first AML evaluation. Its first contribution is the Relaytic-AML architecture: a workspace-centered lab where artifacts, not chat transcripts, carry the truth of an experiment. The second contribution is a concrete multi-agent runtime that separates guidance, scouting, scientific challenge, model building, trace review, and claim review into bounded loops. The third contribution is an evidence-cell contract that makes every reported number traceable to a dataset, split, command, artifact field, budget, operating point, and claim state. The fourth contribution is a release harness that turns local artifacts into tables, figures, source packages, and manuscript claims without letting unsupported interpretations slip through. The fifth contribution is a reproducible handoff surface: a human, local model, or external agent can ask what is known, which artifacts matter, and what safe action comes next.

The scope is deliberate. Relaytic-AML is presented as an evaluation environment and evidence system. Its current public benchmark rows support that systems claim; they are not offered as broad deployment superiority, graph-neural dominance, or equivalence to RevClassify. That boundary is part of the design rather than an afterthought.

The release therefore contributes five concrete mechanisms:

- A local artifact graph for run state, model outputs, traces, metric cells, tables, figures, and source packages.
- A role-scoped agent runtime in which guide, scout, strategist, scientist, builder, trace reviewer, and release governor work through typed artifacts rather than ambient memory.
- Evidence cells that bind each reported number to dataset, split, command, artifact field, budget, leakage posture, operating point, and claim state.
- Governed handoff surfaces for local models, external coding agents, OpenClaw-style skills, Claude/Codex project skills, JSON CLI calls, and Model Context Protocol adapters.
- A release harness that can regenerate the manuscript assets while blocking interpretations not supported by the evidence record.

3 AML Problem Setting

Money laundering and financial crime are not usually visible as a single illegal-looking row. They appear as transaction patterns, entity relationships, and changes in behavior. A transfer can be ordinary in isolation and suspicious in context: many small incoming transfers followed by rapid outward movement, round-number wires without a business explanation, flows to higher-risk locations, repeated transfers among related accounts, weak originator or beneficiary information, or cryptocurrency flows that use anonymity-enhancing services. FinCEN publishes advisories and red-flag material for monitoring practice, while the FFIEC manual includes examples involving funds-transfer behavior, activity inconsistent with a customer profile, cross-border flows, and unusual transactions [Financial Crimes Enforcement Network, 2026, Federal Financial Institutions Examination Council, 2026]. FATF guidance for virtual assets similarly groups red flags around anonymity technology, geography, transaction patterns, transaction size, sender/recipient profiles, and source-of-funds signals [Financial Action Task Force, 2020].

In data terms, AML work starts with events. A transaction record may contain a timestamp, sender, receiver, amount, channel, geography, account or wallet identifiers, device or network context, customer segment, and sometimes balances or counterparties. Fraud can enter through account takeover, mule accounts, front companies, shell-company transfers, trade or invoice activity, digital-asset movement, or a cash-out endpoint. A useful system therefore has to reason over both rows and relationships: what happened before, which entities are connected, whether the flow is unusual for the customer, whether a new counterparty receives many small transfers, and what a human reviewer would need to see.

Relaytic-AML operationalizes that setting in four steps:

- It first freezes the source and task contract: what data is local, which columns are allowed, what target is being modeled, and which split rule prevents future information from leaking backward.
- It then builds time-aware and graph-aware evidence: prior-step history for destination entities, structural counterparty features, suspicious-subgraph context, typology posture, and delayed-label or review-budget artifacts where available.
- It runs baseline and competitive candidate models under explicit budgets, chooses thresholds only on validation evidence, and reports fixed test metrics with leakage posture attached.
- It connects scores to operations by asking which transactions or entities would enter a review queue, what precision and recall look like under a bounded analyst budget, and whether the result can support a paper, product, or compliance claim.

A simple example makes the point. In a mobile-money style dataset, a fraud pattern may look like a transfer into a destination account followed quickly by cash-out. A naive model can accidentally learn from balance columns that reveal the simulator’s internal bookkeeping after the event. Relaytic therefore records those fields as forbidden for the paper row and instead uses features that would be available at decision time, such as prior-step destination behavior. In a blockchain graph, the row is less like a bank transfer table and more like a directed transaction network over time. Relaytic therefore keeps graph provenance and subgraph boundaries visible before treating a score as evidence.

4 Relaytic Design Thesis

The central problem is legibility under constraint. A trained reader can understand PR-AUC, temporal splits, graph features, and review budgets, but that reader cannot know the state of a local experiment unless the system makes it legible. Relaytic-AML is built around the view that an evaluation environment should expose its state in the same way a model exposes its metrics: through stable artifacts, not through informal memory.

The design thesis is that local-first evidence can make agent-assisted AML research more reliable if three invariants hold. First, the local artifact graph must remain the authority for data posture, split decisions, metric values, model artifacts, and release state. Second, agent assistance must be role-scoped: a guide may explain, a scout may inspect, a scientist may challenge, a builder may execute, and a release governor may block a claim, but none of them should silently overwrite the evidence record. Third, every public interpretation of a result must be bounded by what the evidence cell can actually support.

This paper evaluates whether the current Relaytic-AML implementation makes those invariants concrete. The benchmark rows are useful because they exercise the architecture, not because they settle AML detection. The more important question is whether a human, a local model, or an external coding agent can enter the workspace, understand what is known, see what remains blocked, and choose a safe next action without private rows leaving the local boundary.

5 Local-First Agent Architecture

Relaytic is designed around one control rule: the user’s workspace is the authority. Raw and licensed data stay local by default. Run directories, manifests, traces, metric cells, model files, and paper assets form the durable record. Semantic caches, memory indexes, and LLM summaries are derived views. This is different from a remote-first agent that sends private rows to a hosted planner and later reconstructs provenance from a conversation.

The system is agentic, but the roles are concrete. They are small jobs with bounded permissions and visible outputs. A scout inspects source posture and split risk. A strategist turns the investigation into an executable task contract. A scientist challenges baselines and ablations. A builder executes a controlled run. A trace reviewer reconstructs decisions, branches, and claim packets. An evidence reviewer can reject an interpretation even when the model score looks attractive. A guide explains the current state to a human or exports a redacted context pack to another model.

This makes Relaytic host-neutral. OpenClaw can consume the checked-in Relaytic skill notes. Claude Code can use the project-local agent and Model Context Protocol (MCP) configuration. Codex-style skill environments can use the same skill contract. Any host can also use the command-line interface (CLI) JSON surfaces. The important privacy rule is the same across all of them: external agents should receive artifact references, aggregate metadata, commands, and redacted context packs by default, while raw rows and licensed files stay inside the governed workspace unless the operator explicitly changes policy.

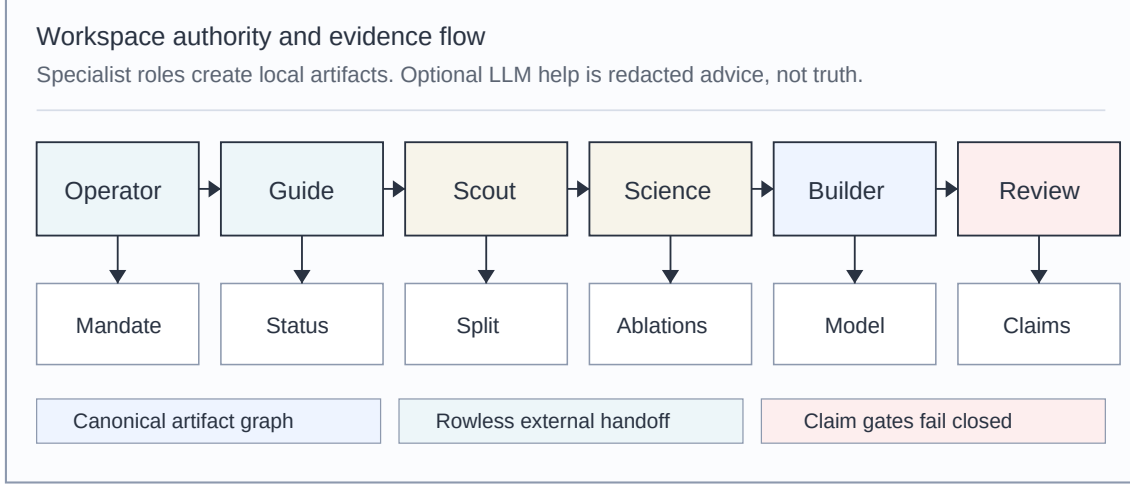
Role	Responsibility	Boundary and outputs
Operator and mandate owner	Sets goals, constraints, privacy posture, and stop/continue preferences.	Mandate, policy, permission, and next-action artifacts. Data can stay on a controlled local machine, server, or cluster.
Guide and assist layer	Answers where the run is, what artifacts matter, and which action is safe next.	Guide payloads, assist turns, status, and context packs. Optional LLM help is advisory and redacted by default.
Scout and task-contract agents	Inspect source posture, target semantics, split validity, and leakage risk.	Dataset registry, source manifests, split contracts, and task reports. Work from staged local snapshots.
Scientist and challenger agents	Propose baselines, ablations, shadow candidates, and failure explanations.	Experiment registry, scorecards, ablations, and shadow-trial reports. Candidate work is budgeted and permission-bound.
Builder and search controller	Execute reproducible model/search plans and select thresholds on validation evidence.	Run directories, model artifacts, search traces, and operating-point records. Optional adapters are versioned.
Evidence and release governors	Decide how evidence may be described and which claims stay blocked.	Metric cells, claim boundaries, release notes, and source-bundle audits. Gates fail closed on unsupported interpretation.
External agents or LLMs	Consume exported context, propose repairs, or continue work through stable surfaces.	External context packs, handoff reports, and reproducible commands. Rowless/redacted by default unless policy grants more.

Two design choices carry most of the system. First, important work produces a local artifact that another human or agent can inspect. Second, optional intelligence is subordinate to the artifact graph. A local LLM may help phrase guidance, and a frontier model may suggest repairs, but neither becomes the source of truth unless its proposal is converted into a reproducible local artifact.

6 Agent Runtime, Loops, and Harness

Relaytic’s technical core is an artifact-first agent loop. Each specialist observes the local run state, reads the contracts it is allowed to read, decides a bounded next action, executes deterministic or advisory logic, writes a typed artifact, and records enough trace information for another process to audit the step. The loop is intentionally more constrained than an open-ended chat agent. It is engineered so that progress survives process restarts, model changes, and external review.

The scout loop is deterministic-first. It converts ingestion metadata, quality checks, stationarity



Benchmarks exercise this architecture; they do not replace the local-first artifact and claim-control thesis.

Figure 1: Relaytic-AML local evidence loop. Specialist roles write local artifacts; external handoff is rowless by default and claim gates fail closed.

heuristics, target-risk signals, and column-name risk scoring into an investigation record. The strategist loop then converts that investigation state into a builder handoff: task type, target field, split route, metric family, candidate steps, and unresolved assumptions. When an advisory model is available, its output is treated as a recommendation attached to the same artifact trail. It does not replace the deterministic contract.

The modeling harness follows the same pattern. Dataset registry artifacts describe what source is being used and what cannot be claimed from it. Split contracts define the temporal or graph partition before model selection. Candidate runners write search traces, validation decisions, calibration records, selected operating points, and test metrics. The release layer reads those outputs as evidence cells. It does not trust a score unless the score can be tied back to the split, command, budget, artifact field, and claim boundary that produced it.

The implementation has six load-bearing loops. The investigation loop prevents model work from starting before the task is coherent. The planning loop turns investigation state into an executable builder contract. The model/search harness separates quick checks, baselines, competitive search, calibration, threshold selection, and fixed test reporting. The trace loop materializes branch choices, tool calls, claim packets, and replay reports. The guide/assist loop makes local state navigable for humans and external agents. The release harness converts evidence cells into paper assets and blocks manuscript claims that exceed the evidence record.

The key technical point is that the agents are not only prompt roles. They are state machines around a governed artifact graph. The architecture favors typed files, schemas, replayable commands, and conservative gates because those are the objects a reviewer, engineer, or future agent can actually inspect. In AML, that matters more than a fluent explanation. A fluent explanation is useful only after the evidence state is already coherent.

6.1 Execution Contract and System Evidence

A full Relaytic run follows a fixed execution contract. The mandate records the user’s goal, constraints, and data-movement posture. Intake and scouting materialize source posture, task

semantics, candidate target fields, leakage warnings, and split recommendations. Planning converts those findings into a builder handoff. Execution writes candidate results and operating-point artifacts. Trace review records decisions, branches, tool use, and claim packets. The release harness then decides what can appear in a table, figure, manuscript claim, or external handoff.

The practical effect is that an agent cannot simply assert that a result is valid. It has to leave behind a path that other surfaces can read. A guide response, an assist turn, a mission-control view, a paper table, and a redacted context pack all read from the same artifact graph. This is why Relaytic is local-first in more than a privacy sense: the local state is also the arbitration layer for meaning.

The run path is deliberately simple: mandate and policy define the authorized work envelope; source and task scouting freeze data posture before model search; planning turns that state into a builder contract; search writes candidate traces and validation decisions; trace review makes the choices replayable; release and handoff promote only interpretations supported by local evidence.

Algorithm 1 shows the specialist loop in implementation terms. The important feature is not that every specialist uses the same model. The important feature is that every specialist must pass through the same artifact, gate, and trace discipline.

Algorithm 1: Artifact-first specialist loop

```
input: local run directory, role contract, privacy policy
while the run still needs work:
    read only the artifacts allowed for this role
    summarize the current evidence state
    propose one bounded next step

    if advisory model help is enabled:
        send a redacted, rowless context pack
        attach the returned note as advice, not authority

    check the proposal against policy, budget, and claim contracts
    if the proposal is allowed:
        write the typed artifact
        append a trace span linking inputs to outputs
    else:
        write a blocker artifact with the rejected reason
        append a trace span linking inputs to the blocker

    refresh the run summary so humans and agents see the same state
```

Algorithm 2 is the corresponding gate for a reported metric. It prevents a high metric value from becoming a stronger paper claim than the evidence cell supports.

Algorithm 2: Evidence-cell claim gate

```

input: candidate metric, dataset contract, split contract, release policy
require dataset identity, split rule, command, artifact field, metric value
require budget tier, leakage posture, operating-point rule, and source refs

if any required field is missing:
    mark the row blocked and name the missing evidence
elif split or leakage checks fail:
    mark the row blocked and prevent table promotion
else:
    assign the narrowest valid claim state
    attach limitation notes and future unlock conditions
    emit the metric cell, allowed claim, blocked claims, and paper refs

```

A simplified PaySim evidence cell contains the dataset identity (`paysim_temporal_transaction_fraud`), a chronological split contract, test PR-AUC 0.638773, a competitive budget tier, the leakage posture `prior_step_destination_history_only`, a supporting-only claim state, and source artifacts for the benchmark manifest, budget contract, and claim gate.

This schema-like record is the difference between a result and an anecdote. It lets a reviewer challenge the dataset, the split, the metric, the budget, the leakage posture, or the claim state without reconstructing the run from memory.

The PaySim row is the clearest example of the evidence-cell path. The source contract identifies a synthetic mobile-money transaction-fraud task. The split contract orders records by simulator step. The leakage contract excludes simulator balance fields that can reveal after-event information, then allows prior-step destination-history features. The model-search contract separates a baseline budget from a competitive budget. The threshold contract chooses operating points on validation evidence and applies them unchanged to the test partition.

That path changes the fixed-test PR-AUC from 0.331345 in the baseline run to 0.638773 in the competitive run. At the selected review budget, the same evidence cell records precision 0.703336 and recall 0.471584. The result is meaningful because the improvement is tied to a documented modeling change under the same split and metric contract. It remains bounded because PaySim is synthetic, so the claim state is supporting temporal-fraud evidence rather than real-bank AML performance.

This is the pattern Relaytic is meant to enforce: useful evidence is preserved, the modeling work that created it is inspectable, and the stronger interpretation is blocked until the data and protocol justify it.

7 Current Frontier Context

The current AML frontier is not a single leaderboard. It is a set of pressure points: real graph scale, realistic entity behavior, temporal drift, operational throughput, and reliable agent-assisted research. Relaytic-AML is designed as infrastructure around those pressure points, not as a replacement for detector papers.

Recent agentic machine-learning work also treats external state, benchmark environments, and research validity as first-class objects. SkillOpt frames agent skills as trainable external state with validation-gated edits [Yang et al., 2026]. MLR-Bench evaluates whether agents can produce valid open-ended machine-learning research rather than only fluent reports [Chen et al., 2025]. PaperBench studies full-paper reproduction by agents and motivates separating fluent paper artifacts from executable reproduction evidence [Starace et al., 2025]. RE-Bench evaluates frontier research-engineering agents against human experts in realistic open-ended environments [Wijk et al., 2025]. Relaytic-AML follows the same broader movement toward executable research state, but applies it to AML-specific data custody, review queues, and release claims.

PaySim is a synthetic mobile-money simulator used here because public transaction-fraud data is scarce and privacy-sensitive [Lopez-Rojas et al., 2016]. It is useful for temporal transaction-fraud workflow evaluation, but synthetic evidence cannot by itself establish hard real-bank AML performance.

The Elliptic Bitcoin dataset introduced a public transaction graph with more than 200K transaction nodes, 234K directed payment-flow edges, and 166 node features across 49 time steps [Weber et al., 2019]. For this paper, it motivates treating graph evidence as something to audit against strong simpler baselines rather than assume superior.

Elliptic2 shifts the public AML benchmark center toward subgraph learning, with 121,810 labeled subgraphs inside a background graph of roughly 49M node clusters and 196M edge transactions [Bellei et al., 2024]. RevTrack and RevClassify provide reference context for sender and receiver information around a subgraph [Song et al., 2024]. These works motivate Relaytic-AML’s modern-context and limitation track, but they do not make the current Relaytic Elliptic2 row a performance contribution.

Recent AML graph work raises the bar beyond the current Relaytic evidence rows. TransXion frames benchmark realism around profile-aware simulation, richer entity attributes, non-template illicit synthesis, and out-of-character behavior [Chen et al., 2026]. LineMVGNN and ExSTraQt focus on directed money flow, edge-aware graph views, and quasi-temporal transaction representations [Poon et al., 2026, Tariq and Hassani, 2026]. BlazingAML treats throughput and multi-stage graph mining as a systems problem [Ye et al., 2026]. Continual graph-learning reviews emphasize drift, adaptation, class imbalance, and changing laundering behavior [Deprez et al., 2025]. These papers point to a frontier where realism, scale, graph structure, time, and operations are inseparable. Relaytic-AML is complementary to those efforts: it does not claim detector parity with them, but tries to make dataset posture, split validity, budget, limitations, and release claims auditable.

The paper also follows broader machine-learning documentation and reproducibility practice. Datasheets for Datasets and Model Cards argue for explicit dataset and model reporting [Gebru et al., 2021, Mitchell et al., 2019]. The NeurIPS reproducibility program highlights the need for code, data, and checklist discipline in machine-learning research [Pineau et al., 2021]. Recent work on machine-learning research agents warns that coherent papers can still contain invalidated experiments, which reinforces the need for executable artifacts, reproducible commands, and explicit claim boundaries [Chen et al., 2025, Starace et al., 2025].

Adjacent practice	What it handles well	Gap for AML evaluation
Experiment tracking	Runs, metrics, artifacts, lineage, and model versions.	A metric can still be separated from task validity, privacy posture, review capacity, and public interpretation.
Data validation and monitoring	Schema checks, drift, quality rules, and production signals.	Validation often stops before model-search budgets, operating-point choices, and manuscript claims are tied together.
Workflow orchestration	DAG execution, retries, scheduling, and dependency management.	Execution graphs usually do not decide whether evidence is strong enough for a claim or a safe next action.
AutoML and benchmark suites	Model-family search, hyperparameter optimization, and leaderboard comparison.	Score search can obscure leakage posture, budget spent, review-queue utility, and whether a result may be generalized.
General agent frameworks	Tool use, planning, memory, and host integration.	Agent fluency can outpace evidence unless state, permissions, redaction, traces, and claim gates are explicit.

The contribution sits in the gap between these categories. Relaytic-AML does not replace experiment tracking, validation, orchestration, AutoML, or agent frameworks. It binds them into a local evidence loop where a result, its budget, its review context, its privacy posture, and its allowed interpretation stay connected.

8 Methodology: Evidence Cells, Gates, and Budgets

Relaytic-AML treats a metric as an evidence cell, not as a free-standing score. An evidence cell records the dataset identity, split contract, execution command, run or artifact reference, metric value, budget tier, leakage posture, operating-point rule, claim state, and limitation notes. A reported row is accepted only when those fields are present and internally consistent.

This creates a small theory of evidence for the system. A metric becomes scientific evidence only after it is attached to provenance, a comparison budget, and an interpretation boundary. Provenance answers where the number came from. Budget answers how much modeling effort was spent before reporting it. The interpretation boundary answers what the number is allowed to mean in public. Removing any one of those pieces turns the row back into an anecdote.

The claim gate is intentionally conservative. It can mark a row as supporting evidence, modern context, baseline-only evidence, or blocked evidence. In the current public version, no row is headline-eligible. The conservative posture is part of the evaluation contract: a strong-looking number is only useful if the system can also state what the number is allowed to mean.

The budget ladder separates quick engineering checks from serious evidence. Smoke runs test that commands and artifacts exist. Baseline runs establish conservative rows. Competitive runs add stronger feature families, validation-only threshold selection, calibration, and model search.

Frozen reporting runs preserve the transformation from experiment to table, figure, and public claim. This prevents a weak first pass from being quietly promoted into a strong paper claim and also prevents a successful search from hiding how much effort was spent.

For an external reader, the important idea is simple: every number has a trail, and every claim has a boundary. The trail helps another person reproduce or challenge the result. The boundary says whether the result is a benchmark observation, an operational estimate, a limitation, or a claim that should not be made yet.

9 Evidence Operating Layer

The local-first architecture becomes concrete through an evidence operating layer. Relaytic-AML is not only a model runner. It coordinates source posture, task semantics, split discipline, model search, decision thresholds, review queues, artifacts, trace review, and claim boundaries.

The table below is deliberately more useful than another list of agent names. It shows the evidence substrate a reader should expect to find behind any serious Relaytic result.

Layer	What It Records	Why It Matters
Source and task contracts	Dataset access, target semantics, benchmark posture, and split rules.	Lets a reader judge task validity before judging a score.
Execution and search artifacts	Candidate families, budgets, calibration, thresholds, selected runs, and search traces.	Makes model development inspectable rather than anecdotal.
AML domain artifacts	Entity graphs, typology posture, review queues, delayed labels, and case evidence.	Connects model output to analyst workflow instead of treating rows in isolation.
Evidence ledgers	Metric cells, tables, figures, limitations, and commands.	Keeps every public number tied to local source artifacts.
Claim boundaries	Allowed claims, blocked claims, limitation notes, and future unlock conditions.	Prevents metric values from becoming unsupported claims.
Trace and replay artifacts	Runtime spans, branch graphs, tool logs, claim packets, and adjudication scorecards.	Lets another reviewer reconstruct how Relaytic reached the state.
Handoff surfaces	Guide, status, assist, mission control, and redacted context export.	Lets humans and external agents continue without guessing hidden state.

This operating layer is useful even when a result cannot support a stronger interpretation. The row is not thrown away. It becomes a structured research state with a reason, an artifact reference, and a repair path.

10 What Relaytic-AML Is For

The intended user is a person or team that has data, a risky modeling question, and a need to know whether the evidence is strong enough to act on. That includes a bank team comparing a new model against an incumbent queue, a fraud group exploring a new dataset, a researcher testing a benchmark protocol, or an external agent trying to continue a run without seeing private rows.

The practical capabilities are deliberately operational. Private or licensed data can stay outside the public repo while hashes, access posture, and evidence artifacts remain inspectable. Tables and figures are tied to local JSON and model artifacts rather than loose notes. The guide, scout, scientist, builder, trace reviewer, and claim reviewer have different responsibilities. Quick checks, baseline evidence, competitive runs, and frozen reporting are kept separate. Review-budget precision, recall, false-positive burden, and case-packet completeness sit next to model metrics. External agents receive redacted state, artifact references, limitations, and reproducible commands rather than private rows by default.

For other agents, Relaytic behaves like a local evidence service rather than a private-data proxy. An OpenClaw workflow can read the Relaytic skill and call the same command surfaces. A Claude Code session can use the project-local agent notes and MCP configuration. A Codex or similar project-skill host can follow the checked-in skill contract. In all cases the safe handoff is artifact-first: the agent sees the run summary, selected artifact references, aggregate metrics, limitations, and reproduction commands. It does not need raw private rows to decide whether the next action is a rerun, a leakage repair, a stronger baseline, a data-acquisition step, or a claim edit.

This is why the project moved from a broad Relaytic system toward Relaytic-AML as the flagship story. The general architecture still matters, but AML gives it a sharper test. It forces the system to handle rare events, temporal splits, graph context, human review limits, privacy, and interpretation boundaries at the same time.

Companies could use this kind of lab to challenge incumbent rules or models on the same review queue, evaluate whether a new dataset is worth deeper investment, audit whether a vendor comparison is fair, and prepare evidence packs for compliance or model-risk review. Engineering leaders could use it to test whether an agent-assisted machine-learning workflow is actually governable: whether the system knows its state, exposes its assumptions, records its choices, and blocks interpretations not supported by recorded evidence. Researchers could use it to ask whether an AML result is supported, blocked, or mainly useful as a limitation.

11 Evaluation Environment

The current evaluation environment combines public benchmark evidence with local artifact discipline. Dataset registry artifacts define source and access posture. Split contracts define chronological, graph-snapshot, or subgraph partition rules. Benchmark runners produce model and operating-point artifacts. Tables and figures then read from those artifacts.

The current public evidence uses three tracks. PaySim is a synthetic temporal proxy. Elliptic is temporal graph-feature evidence. Elliptic2 is retained as modern subgraph context and limitation evidence because a faithful RevClassify reference-protocol reproduction has not yet been completed locally.

The environment follows three design rules. Local artifacts are the source of truth. Validation

selects models, thresholds, and operating points before fixed test evaluation. Blocked evidence stays visible because hiding failed or incomplete tracks makes both human and agent-assisted research less scientific.

11.1 System Evaluation

The benchmark rows are not the only evaluation target. Relaytic itself has to behave like a usable research instrument. The current paper therefore treats system behavior as part of the evidence: whether a user or another agent can recover state, find the right artifacts, understand what is blocked, and continue without private rows leaving the workspace.

The current implementation emits five kinds of system evidence. State recovery is covered by guide payloads, status fallback, assist turns, mission-control summaries, run summaries, artifact shortlists, and next-step options generated from local artifacts. External-agent handoff is covered by rowless context packs, OpenClaw notes, Claude/Codex skill contracts, JSON CLI surfaces, and MCP surfaces. Metric auditability is covered by evidence cells, metric-cell audits, command ledgers, budget contracts, and release manifests. Agent traceability is covered by runtime spans, specialist traces, branch graphs, claim packets, replay reports, and adjudication scorecards. Claim boundaries are covered by release-gate records, limitation notes, and evidence-backed allowed or blocked claim lists.

The release pack measures part of the system behavior directly. The target is not user satisfaction. The target is a more basic property that an evaluation lab should have: a reader or external agent should be able to enter the repository, find the paper evidence, recover the current state, trace a number back to its source, and see why a stronger claim remains blocked.

The current pack contains 22 required deterministic checks. Within it, 11 reader and external-agent tasks cover repository navigation, cross-platform reproduction, metric provenance, baseline-versus-competitive comparison, claim-boundary recovery, rowless handoff. These tasks are intentionally concrete. They ask whether the README separates the general Relaytic platform from the Relaytic-AML paper path, whether Windows and macOS/Linux reproduction commands are visible, whether the PaySim PR-AUC cell carries dataset/split/command/artifact/budget/leakage/claim provenance, whether PaySim baseline and competitive budgets are comparable, whether Elliptic2 is recoverable as modern context rather than a performance contribution, and whether an interrupted run can be exported to another model without raw rows or private paths.

This matters because the paper’s central claim is about controlled evidence, not only detector performance. A strong PR-AUC with no recoverable provenance would be weak evidence for this paper. Conversely, a blocked Elliptic2 row is still useful when the system can explain exactly why it is blocked and which future evidence would change that state.

The evaluation also checks the local-first handoff contract. Relaytic exports a rowless external-agent context pack from local artifacts, verifies that raw rows are absent, records redactions, and exposes safe next actions plus tool discovery. Optional local large-language-model phrasing remains advisory in the evaluated fixture; the truth-bearing state is the artifact graph.

All required checks currently pass. The result should be read narrowly but seriously: Relaytic demonstrates deterministic navigation, provenance recovery, partial-run recovery, rowless hand-off, optional-LLM containment, and fail-closed claim gating. It does not claim a controlled human-subject result, analyst-hour savings, production deployment, or autonomous external-agent performance improvement.

The repository publishes the task-level system evaluation, aggregate behavior evaluation,

partial-run recovery check, rowless handoff check, claim-gate case studies, and fail-closed manifest as machine-readable evidence. The README maps those generated reports to concrete filenames for readers who want to audit the JSON.

The PaySim competitive row illustrates the contract. The run uses a chronological split, excludes forbidden balance fields, builds prior-step destination history, selects candidates on validation evidence, and freezes the test operating point. The resulting PR-AUC is reported because the evidence cell is complete. It is not promoted to a real-bank AML claim because the data is synthetic. That refusal is a system result: Relaytic preserved the useful score while preventing a stronger but unsupported interpretation.

12 Benchmark Protocol

The benchmark protocol is deliberately subordinate to the architecture. Its purpose is to check whether Relaytic can produce traceable evidence, respect split and leakage contracts, expose operational assumptions, and block unsupported claims. A single score does not define the value of the system.

The public rows separate baseline and competitive evidence. Competitive rows use stronger feature families, candidate search, calibration, and validation-only operating-point selection. The point is to make modeling effort visible and to keep future reruns comparable rather than letting a single metric dominate the evidence record.

The evidence summary below should be read as a claim-boundary table as much as a performance table. The numbers matter, but the posture column is what stops them from being overstated.

Evidence row	Metric	Value	Claim posture
PaySim baseline	test PR-AUC	0.331345	baseline-only
PaySim competitive	test PR-AUC	0.638773	supporting-only synthetic temporal proxy
PaySim competitive	precision at review budget	0.703336	supporting-only
PaySim competitive	recall at review budget	0.471584	supporting-only
Elliptic graph-feature	test PR-AUC	0.668756	supporting-only graph-feature evidence
Elliptic graph-feature	precision at review budget	1	supporting-only
Elliptic graph-feature	recall at review budget	0.056604	supporting-only
Elliptic2 context	official-partition PR-AUC mean	0.94324	modern context only
Elliptic2 context	official-partition PR-AUC std	0.000882	modern context only
RevClassifyDS reference	published PR-AUC	0.974	reference context, not parity

The PaySim row changes from the baseline result (0.331345) to the competitive result (0.638773). That is a substantial improvement inside the synthetic temporal-fraud contract, and it is the clearest current sign that Relaytic’s workflow is doing useful modeling work rather

than only post-processing a baseline report. The likely driver is the combination of leakage exclusion, prior-step destination-history features, validation-selected model choice, and explicit competitive search. The same row still cannot become a real-bank AML claim because the source is synthetic.

The Elliptic row (0.668756) is different. It is useful because it keeps graph-source and structural-feature evidence under a chronological snapshot protocol, but the graph-feature lift over source-only evidence is small. It supports Relaytic’s provenance and split discipline; it does not show that Relaytic has solved graph-native AML detection.

The Elliptic2 context row (0.94324 +/- 0.000882) is high in absolute terms, but the recorded RevClassifyDS reference is 0.974. That makes it valuable as modern-context pressure, not as a parity or superiority result. The system’s behavior here matters: Relaytic preserves the evidence and also blocks the stronger interpretation.

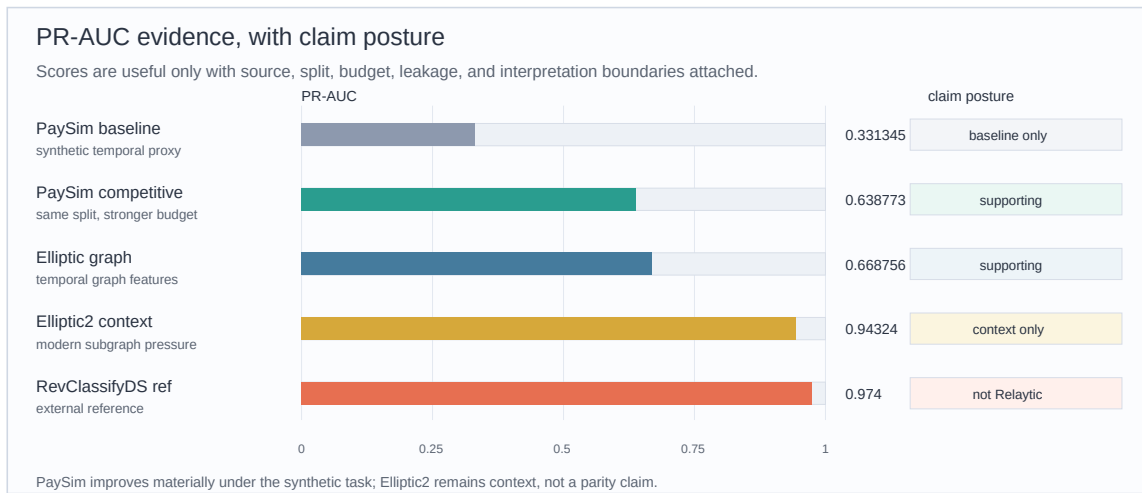


Figure 2: PR-AUC evidence rows. PaySim improves under the synthetic temporal proxy, while Elliptic2 remains below the RevClassifyDS reference and is not a parity claim.

Figure 2 visualizes the same distinction. PaySim is the clearest improvement row because the competitive result materially improves on the baseline under the same synthetic task. Elliptic and Elliptic2 answer narrower questions: whether the environment can carry graph evidence and whether it can keep a strong modern-context row separate from a blocked state-of-the-art claim.

Figure 3 translates scores into review-queue behavior. PaySim has high review-budget precision (0.703336) with much lower recall (0.471584), which is typical when only a small top-ranked queue is inspected. Elliptic shows a similar asymmetry: precision 1, recall 0.056604. This is operationally important because an AML team would not read PR-AUC alone. They would ask whether the top of the queue is useful, how much fraud remains outside the reviewed set, and whether the threshold was chosen without touching the test set.

13 Results

The strongest current empirical result is PaySim. It is not a headline AML result, but it is a useful systems result: Relaytic starts from a modest baseline, expands the feature/search budget, excludes leakage-prone fields, selects on validation evidence, and produces a much stronger fixed-

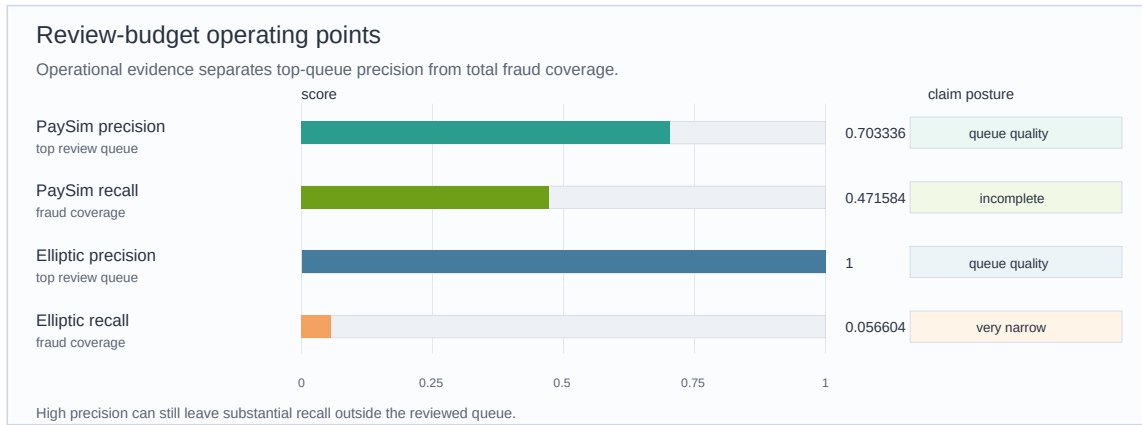


Figure 3: Review-budget operating points. High top-queue precision with lower recall shows useful but incomplete analyst coverage under a bounded review budget.

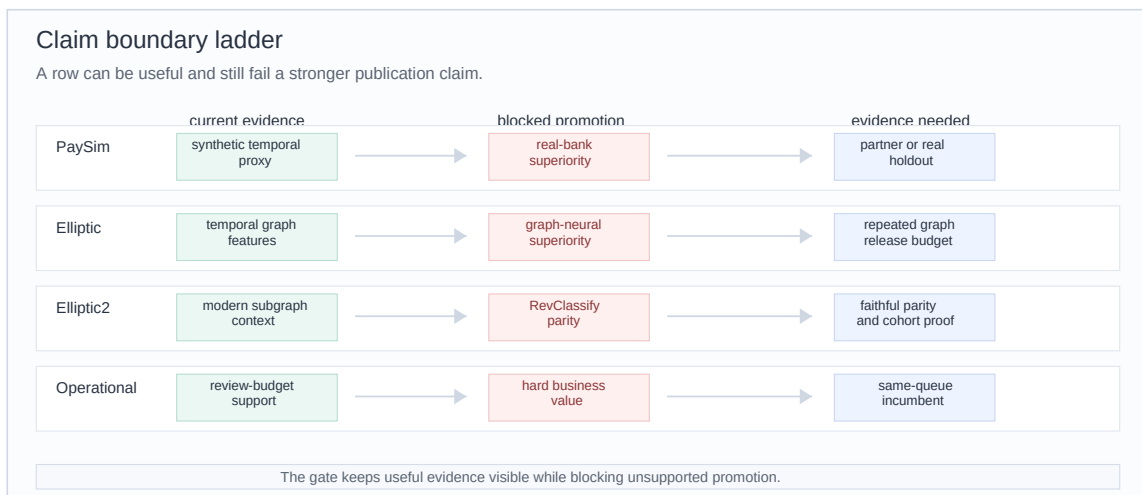


Figure 4: Claim boundary ladder. Current evidence is preserved while stronger AML, graph-neural, RevClassify-parity, and business-value claims remain blocked.

test PR-AUC under the same public synthetic task. This is the type of controlled improvement an evaluation lab should surface before advancing stronger claims.

The Elliptic result is more cautious. It shows that Relaytic can ingest and evaluate temporal graph-feature evidence with provenance attached, but the current public row does not prove that Relaytic has a superior graph learner. That is an honest limitation and also a useful next target: future work should test stronger graph-native families, repeated budgets, and stricter graph protocols.

The Elliptic2 result is a warning against overclaiming. The number is high in absolute terms, but it remains below the recorded RevClassifyDS reference and the local reproduction path does not yet prove reference-protocol parity. Relaytic keeps that distinction in the evidence record instead of converting it into a stronger claim.

The broader result is the behavior of the environment. Relaytic can preserve a useful score while refusing an unsupported interpretation. In financial-crime machine learning, that refusal is part of scientific and operational discipline. The gate decisions are concise: PaySim is supporting synthetic temporal-fraud evidence, Elliptic is supporting graph-feature evidence, Elliptic2 is modern context and limitation evidence, and the operational layer is review-budget support rather than proven analyst-hour savings.

14 Discussion

The practical value of Relaytic-AML is not that it replaces a compliance platform or reports the highest value in a benchmark table. Its value is that it gives risk, fraud, or AML teams a local evidence lab for unknown datasets, incumbent challenges, review-budget tradeoffs, leakage checks, and claim discipline. A team evaluating a new dataset can inspect whether the result is a model win, an environment win, a proxy-only result, or a blocked claim.

For research teams and agentic machine-learning workflows, the same structure is a guard against coherent but invalid experiments. External agents can read the structured artifacts, see the claim posture, and propose the next benchmark action without inferring hidden state from prose. The central contribution is artifact discipline: Relaytic-AML demonstrates that an ambitious evaluation system can keep progress and restraint in the same loop.

The same structure also helps when an experiment changes direction. A row can tell a team that a dataset is proxy-only, that a split needs repair, that a graph-native candidate must beat strong tabular features, that a review-queue claim needs a same-queue incumbent, or that a public sentence is stronger than the evidence permits. In high-risk financial-crime settings, those answers save time because they turn ambiguity into the next concrete engineering step.

An organization would use Relaytic-AML not as a replacement for domain expertise, but as an evaluation layer that makes difficult assumptions explicit: what data was allowed to move, what split was used, what budget was spent, whether the threshold was chosen on validation evidence, what a reviewer would see, and which interpretation the evidence can support.

15 Limitations

The current public evidence is not a deployment validation. PaySim is synthetic mobile-money evidence, so it cannot establish real-bank AML superiority. The Elliptic row is a temporal graph-feature result, not proof that a graph-neural model is better. Elliptic2 is modern context, not a

Relaytic performance contribution, because faithful reference-protocol reproduction and cohort equivalence still need more work.

The operational evidence also remains early. The review-budget rows are useful for showing how Relaytic connects model output to analyst capacity, but they do not prove analyst-hour savings against an incumbent queue. A stronger version of this work should include complete case packets, a same-queue incumbent comparison, and a partner-approved or otherwise realistic holdout.

The paper therefore argues for Relaytic-AML as a useful local evaluation environment. The current benchmark rows demonstrate the architecture and evidence discipline. They do not close the broader detector question, which should be tested with stronger holdouts, larger search budgets, and partner-grade operational comparisons.

16 Future Work

The next system step is to move beyond deterministic protocol checks into controlled interaction studies. Relaytic should be tested with first-time users, specialist reviewers, and external agents on the same local run states: how quickly they recover the current state, whether they choose the right next action, whether they avoid unsupported claims, and whether the redacted context pack gives another model enough information to propose a useful repair.

The AML evidence should move in parallel. A stronger version needs a partner-approved or otherwise realistic holdout, faithful Elliptic2 reference reproduction, stronger graph-native candidates, continual-learning experiments, and same-queue business-value comparisons. The point is not to replace the current architectural story with a leaderboard story. The point is to give the architecture harder evidence to govern.

A useful next paper would therefore have two coupled evaluations. The detector evaluation would test stronger AML candidates under frozen budgets and leakage-resistant splits. The environment evaluation would use timed, task-based human and agent studies rather than only deterministic protocol checks. That would turn Relaytic’s usability promise into a stronger empirical benchmark while preserving the local-first privacy boundary.

The longer-term goal is still broader than AML. Relaytic should become a general local evaluation laboratory for structured, temporal, and graph machine learning. AML is the current flagship because it forces the system to handle privacy, time, graph context, human review, and claim discipline together.

17 Reproducibility

The code, paper source, figures, tables, and public evidence artifacts are in the Relaytic repository. The repository is larger than the AML paper: Relaytic is the general local-first inference lab, while Relaytic-AML is the flagship edition used here to evaluate financial-crime workflows. A reader who wants to review the paper should start with this manuscript and the README. The broader architecture, interoperability, runtime, build-history, and academy-planning documents are useful context, but they are not all part of the AML benchmark contribution.

The public repo keeps raw private or licensed data out of version control. Where a benchmark requires local data, the command ledger describes the expected local paths and access posture.

A compact Windows PowerShell reproduction path for the public paper assets is:

```
py -3.11 -m pip install -e ".[full]"
py -3.11 -m relaytic.ui.cli release-safety paper-tables --format json
py -3.11 -m relaytic.ui.cli release-safety paper-draft --format json
py -3.11 -m relaytic.ui.cli release-safety paper-dry-run --format json
py -3.11 -m relaytic.ui.cli release-safety paper-system-eval --format json
py -3.11 -m relaytic.ui.cli release-safety paper-release --format json
py -3.11 -m relaytic.ui.cli release-safety paper-arxiv-source --format json
py -3.11 -m relaytic.ui.cli scan-git-safety
```

The same path on macOS or Linux is:

```
python3 -m pip install -e ".[full]"
python3 -m relaytic.ui.cli release-safety paper-tables --format json
python3 -m relaytic.ui.cli release-safety paper-draft --format json
python3 -m relaytic.ui.cli release-safety paper-dry-run --format json
python3 -m relaytic.ui.cli release-safety paper-system-eval --format json
python3 -m relaytic.ui.cli release-safety paper-release --format json
python3 -m relaytic.ui.cli release-safety paper-arxiv-source --format json
python3 -m relaytic.ui.cli scan-git-safety
```

To recheck the system-evaluation claim without rebuilding every table, run only the system-evaluation command and the focused test:

```
py -3.11 -m relaytic.ui.cli release-safety paper-system-eval --format json
py -3.11 -m pytest tests/test_paper_track_p15.py -q
```

On macOS or Linux, replace `py-3.11` with `python3`. The system-evaluation command writes task-level evidence, aggregate behavior evidence, rowless handoff evidence, partial-run recovery evidence, claim-gate case studies, and a fail-closed manifest. The README names the generated JSON files for readers who want to inspect them directly.

For navigation, use the README as the repo map. The repository is larger than the manuscript: the AML paper assets and generated evidence reports are the paper path, while broader architecture, interoperability, runtime, build-history, and planning documents are product context rather than required paper reading.

18 Author Use of AI Assistance

Large-language-model tools assisted with drafting, editing, repository inspection, and consistency checks. The scientific framing, claim boundaries, experimental interpretation, and final manuscript remain the author's work. The tools are not listed as authors.

19 Conclusion

Relaytic-AML should be read as a local-first AML evaluation-lab paper. The current work is valuable because it makes the operating idea concrete. Data stays locally governed. Specialist

roles create inspectable artifacts. External agents receive structured context instead of hidden state. Claim boundaries keep manuscript claims aligned with evidence. The result is a system where model development, privacy, agent assistance, and paper claims are connected by one artifact graph.

The right test for this version is therefore whether the repository makes the current evidence easier to inspect, easier to challenge, and less likely to be overstated. That is the claim this paper is prepared to make.

References

- Claudio Bellei, Muhua Xu, Ross Phillips, Tom Robinson, Mark Weber, Tim Kaler, Charles E. Leiserson, Arvind, and Jie Chen. The shape of money laundering: Subgraph representation learning on the blockchain with the elliptic2 dataset, 2024. URL <https://arxiv.org/abs/2404.19109>.
- Hui Chen, Miao Xiong, Yujie Lu, Wei Han, Ailin Deng, Yufei He, Jiaying Wu, Yibo Li, Yue Liu, and Bryan Hooi. Mlr-bench: Evaluating ai agents on open-ended machine learning research, 2025. URL <https://arxiv.org/abs/2505.19955>.
- Keyang Chen, Mingxuan Jiang, Yongsheng Zhao, Zeping Li, Zaiyuan Chen, Weiqi Luo, Zhixin Li, Sen Liu, Yinan Jing, Guangnan Ye, Xihong Wu, and Hongfeng Chai. Transxion: A high-fidelity graph benchmark for realistic anti-money laundering, 2026. URL <https://arxiv.org/abs/2604.17420>.
- Bruno Deprez, Wei Wei, Wouter Verbeke, Bart Baesens, Kevin Mets, and Tim Verdonck. Advances in continual graph learning for anti-money laundering systems: A comprehensive review, 2025. URL <https://arxiv.org/abs/2503.24259>.
- Federal Financial Institutions Examination Council. Bsa/aml examination manual: Appendix f, money laundering and terrorist financing red flags, 2026. URL <https://bsaaml.ffiec.gov/manual/Appendices/07>.
- Financial Action Task Force. Virtual assets red flag indicators of money laundering and terrorist financing, 2020. URL <https://www.fatf-gafi.org/en/publications/Methodsandtrends/Virtual-assets-red-flag-indicators.html>.
- Financial Crimes Enforcement Network. Alerts, advisories, notices, bulletins, and fact sheets, 2026. URL <https://www.fincen.gov/resources/advisoriesbulletinsfact-sheets>.
- Timnit Gebru, Jamie Morgenstern, Briana Vecchione, Jennifer Wortman Vaughan, Hanna Wallach, Hal Daume III, and Kate Crawford. Datasheets for datasets. *Communications of the ACM*, 64(12):86–92, 2021. doi: 10.1145/3458723. URL <https://arxiv.org/abs/1803.09010>.
- Edgar Alonso Lopez-Rojas, Ahmad Elmir, and Stefan Axelsson. Paysim: A financial mobile money simulator for fraud detection. In *Proceedings of the 28th European Modeling and Simulation Symposium*, 2016. URL https://www.msc-les.org/proceedings/emss/2016/E_MSS2016_249.pdf.
- Margaret Mitchell, Simone Wu, Andrew Zaldivar, Parker Barnes, Lucy Vasserman, Ben Hutchinson, Elena Spitzer, Inioluwa Deborah Raji, and Timnit Gebru. Model cards for model reporting. In *Proceedings of the Conference on Fairness, Accountability, and Transparency*, pages 220–229, 2019. doi: 10.1145/3287560.3287596. URL <https://arxiv.org/abs/1810.03993>.

- Joelle Pineau, Philippe Vincent-Lamarre, Koustuv Sinha, Vincent Lariviere, Alina Beygelzimer, Florence d’Alche Buc, Emily Fox, and Hugo Larochelle. Improving reproducibility in machine learning research. *Journal of Machine Learning Research*, 22(164):1–20, 2021. URL <https://www.jmlr.org/papers/v22/20-303.html>.
- Chung-Hoo Poon, James Kwok, Calvin Chow, and Jang-Hyeon Choi. Linemvgnn: Anti-money laundering with line-graph-assisted multi-view graph neural networks, 2026. URL <https://arxiv.org/abs/2603.23584>.
- Kiwhan Song, Mohamed Ali Dhraief, Muhua Xu, Locke Cai, Xuhao Chen, Arvind, and Jie Chen. Identifying money laundering subgraphs on the blockchain. In *Proceedings of the 5th ACM International Conference on AI in Finance*, 2024. doi: 10.1145/3677052.3698635. URL <https://arxiv.org/abs/2410.08394>.
- Giulio Starace, Oliver Jaffe, Dane Sherburn, James Aung, Jun Shern Chan, Leon Maksin, Rachel Dias, Evan Mays, Benjamin Kinsella, Wyatt Thompson, Johannes Heidecke, Amelia Glaese, and Tejal Patwardhan. Paperbench: Evaluating ai’s ability to replicate ai research, 2025. URL <https://arxiv.org/abs/2504.01848>.
- Haseeb Tariq and Marwan Hassani. Extracting money laundering transactions from quasi-temporal graph representation, 2026. URL <https://arxiv.org/abs/2604.02899>.
- Mark Weber, Giacomo Domeniconi, Jie Chen, Daniel Karl I. Weidele, Claudio Bellei, Tom Robinson, and Charles E. Leiserson. Anti-money laundering in bitcoin: Experimenting with graph convolutional networks for financial forensics, 2019. URL <https://arxiv.org/abs/1908.02591>.
- Hjalmar Wijk, Tao Roa Lin, Joel Becker, Sami Jawhar, Neev Parikh, T. Broadley, Lawrence Chan, Michael Chen, Joshua M. Clymer, Jai Dhyan, Elena Elicheva, Katharyn Garcia, Brian Goodrich, Nikola Jurkovic, Megan Kinniment, Aron Lajko, Seraphina Nix, Lucas Jun Koba Sato, William Saunders, Maksym Taran, Ben West, and Elizabeth Barnes. Re-bench: Evaluating frontier ai r&d capabilities of language model agents against human experts. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 66772–66832, 2025. URL <https://proceedings.mlr.press/v267/wijk25a.html>.
- Yifan Yang, Ziyang Gong, Weiquan Huang, Qihao Yang, Ziwei Zhou, Zisu Huang, Yan Li, Xuemei Gao, Qi Dai, Bei Liu, Kai Qiu, Yuqing Yang, Dongdong Chen, Xue Yang, and Chong Luo. Skillopt: Executive strategy for self-evolving agent skills, 2026. URL <https://arxiv.org/abs/2605.23904>.
- Haojie Ye, Arjun Laxman, Yichao Yuan, Krisztian Flautner, and Nishil Talati. Blazingaml: High-throughput anti-money laundering (aml) via multi-stage graph mining, 2026. URL <https://arxiv.org/abs/2604.12241>.